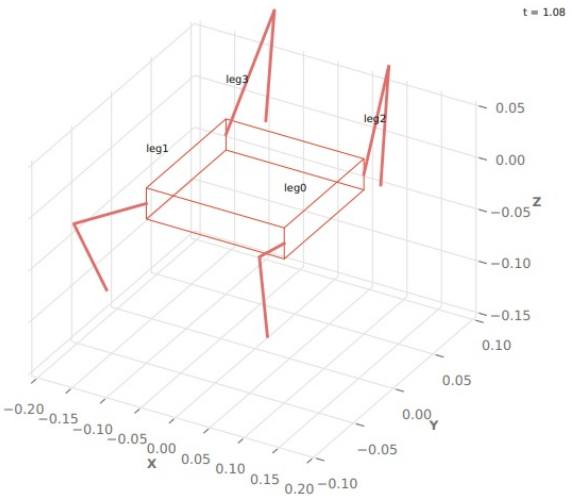
TEL200 - Introduction to Robotics

Group 5



Walking Robot Project

Authors

Andreas Brustad
Johannes Standal
Håkon Bekken

Norwegian University of Life Sciences

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Motion Planning 2
2.2 Path Planning 2
2.3 Localization and Mapping 2

3 Method **2**
3.1 Motion Planning 2
3.2 Motion Primitives 5
3.3 Path Planning 5
3.4 Localization and Mapping 6

4 Results **7**

5 Discussion **7**

6 Conclusions **7**

7 Bibliography **7**

1 Abstract

2 Introduction

This report concerns the design and implementation of a simulated quadruped walking robot. The project covers three main topics: motion planning for coordinated leg movement, path planning for obstacle-free navigation, and localization and mapping for situational awareness. Together, these form the core components of an autonomous robotic system.

2.1 Motion Planning

A quadruped walking robot requires coordinated control of four legs, each kinematically equivalent to a three-joint serial-link manipulator. Gait generation ensures static stability by keeping at least three feet in contact with the ground at all times, dividing each leg's motion into a stance phase and a reset phase. The resulting crawl gait is achieved by assigning each leg a phase-shifted version of the same cyclic foot trajectory.

2.2 Path Planning

For a robot to navigate an environment, it must plan a collision-free path from a start position to a goal. The Probabilistic Roadmap (PRM) method addresses this by constructing a sparse graph of obstacle-free paths through the free space during an offline planning phase. Once built, the roadmap can be reused to answer any start-to-goal query efficiently, making PRM well suited to large-scale and high-dimensional planning problems.

2.3 Localization and Mapping

Autonomous navigation requires a robot to know both its own position (localization) and the structure of its environment (mapping). These two problems are interdependent: a map is needed for localization, and a pose estimate is needed for mapping. When neither is known in advance, they must be solved simultaneously — a problem known as Simultaneous Localization and Mapping (SLAM). Common approaches include filter-based methods such as the Extended Kalman Filter (EKF) and the Particle Filter (PF), as well as graph-based optimization methods.

3 Method

3.1 Motion Planning

A robot leg is kinematically equivalent to a robot arm. For a statically stable quadruped, a three-joint serial-link manipulator is sufficient, since the foot makes point contact with the ground and orientation is not critical Corke, 2023.

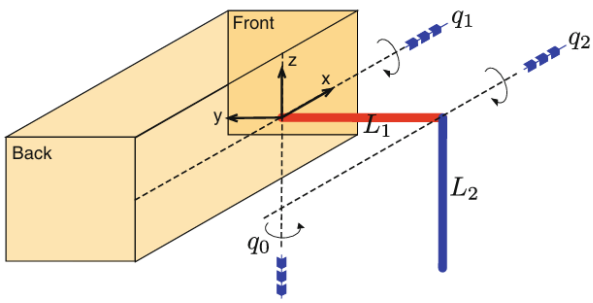


Figure 1: Illustration of frame and axis rotations for the simple leg Corke, 2023

The coordinate frame is defined with the x-axis pointing forward, the z-axis upward, and the y-axis to the left. The leg link lengths are $L_1 = L_2 = -100\,\text{mm}$. The hip has two degrees of freedom: rotation about the z-axis (forward/backward) and rotation about the x-axis (up/down). The knee is offset by L_1 in the y-direction and rotates about the x-axis, while the foot is offset by L_2 in the z-direction. In ETS notation:

$$\texttt{leg} = R_z \cdot R_x \cdot T_y(L_1) \cdot R_x \cdot T_z(L_2) \tag{1}$$

At zero joint angles, forward kinematics yields a foot position of $(0,\ -0.1,\ -0.1)\,\text{m}$, confirming the design.

During the *stance phase*, the foot remains in contact with the ground 50 mm below the hip, moving from 50 mm forward to 50 mm behind the hip to propel the body forward. During the *reset phase*, the foot lifts to 20 mm below the hip, swings forward, and is placed back on the ground. To maintain static stability, at least three feet must remain on the ground at all times, so each leg spends 75% of the cycle in stance and 25% resetting. This coordinated pattern is known as a *crawl gait* (or *wave gait*).

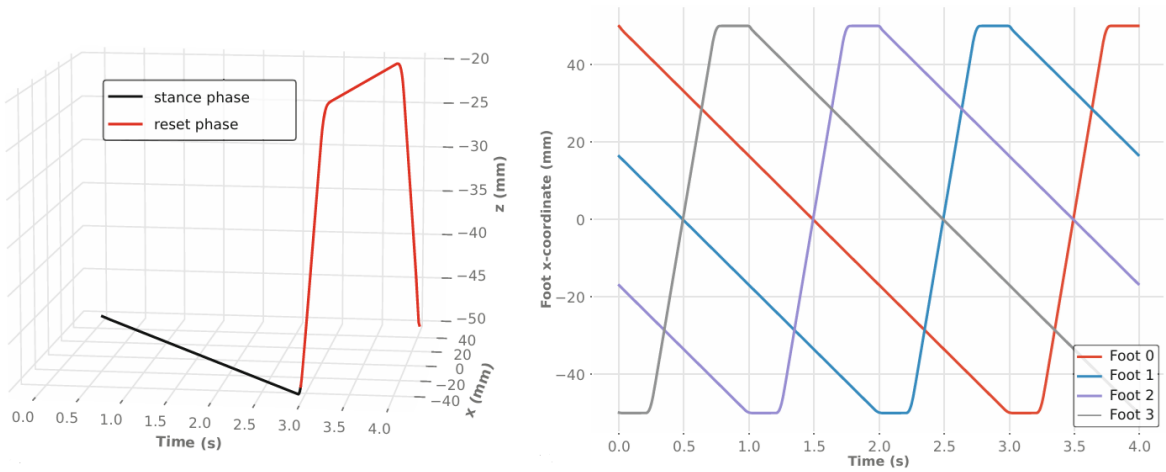


Figure 2: Trajectory of a single foot and motion of each leg Corke, 2023

The foot path is defined by five via points forming a closed cycle:

$$\mathbf{p}_{\text{via}} = \begin{bmatrix} x_f & y & z_d \\ x_b & y & z_d \\ x_b & y & z_u \\ x_f & y & z_u \\ x_f & y & z_d \end{bmatrix} \tag{2}$$

where $x_f = 50\,\text{mm}$, $x_b = -50\,\text{mm}$, $y = -50\,\text{mm}$, $z_u = -20\,\text{mm}$, and $z_d = -50\,\text{mm}$. The path is sampled at 100 Hz using `mstraj` with segment times $[3,\ 0.25,\ 0.5,\ 0.25]\,\text{s}$, giving a 4 s cycle of 400 points. Joint angle trajectories are obtained by numerical inverse kinematics, solving only for end-effector translation:

$$\texttt{mask} = [1,\ 1,\ 1,\ 0,\ 0,\ 0] \tag{3}$$

The robot body has width $W = 100\,\text{mm}$ and length $L = 200\,\text{mm}$. Four leg instances are placed at the body corners using base transforms. Legs on the left side (legs 2 and 3) are rotated by π about the z-axis so they point outward from the body. Each leg runs the same cyclic trajectory with a phase offset of one quarter of the cycle (100 points), achieved via modulo indexing:

$$k' = (k + \phi) \mod N \tag{4}$$

where k is the current time step, $\phi \in \{0, 100, 200, 300\}$ is the phase offset per leg, and $N = 400$ is the total cycle length. For legs on the right-hand side, the sign of the first joint angle is flipped to mirror the motion symmetrically.

For global positioning the robot is modelled as *unicycle*, a common kinematic abstraction for mobile robots moving on a planar surface. The robot pose is fully described by three states:

$$\mathbf{x}_v = (x,\ y,\ \theta)^T \tag{5}$$

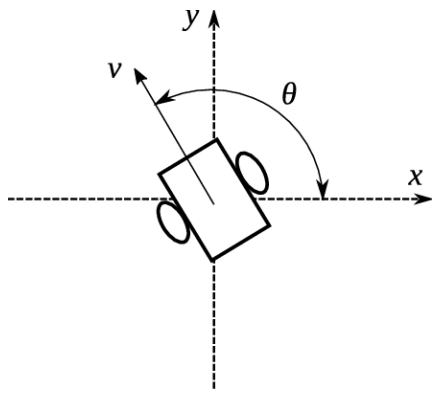


Figure 3: Illustration of pose using unicycle model Li, 2017

where x and y are the position coordinates in the global reference frame and θ is the heading angle relative to the x-axis.

3.2 Motion Primitives

The global motion of the robot is constructed from two discrete motion primitives:

- Forward translation of 10 cm
- In-place rotation of 1°

For forward motion, one complete gait cycle produces a net displacement of 10 cm in the heading direction. For rotation, one gait cycle corresponds to a net change of 1°in orientation. Each primitive corresponds to one complete gait cycle in joint space. During forward motion, the robot base is incrementally translated along its current heading direction according to

$$x_{k+1} = x_k + \Delta s \cos\theta_k, \qquad y_{k+1} = y_k + \Delta s \sin\theta_k \tag{6}$$

where Δs is the incremental displacement per simulation step.

For rotation, the heading angle is updated as

$$\theta_{k+1} = \theta_k + \Delta\theta \tag{7}$$

Utilizing these motion primitives, we can achieve planar motion by combining forward and rotational steps. However since both translation and rotation are reduced to certain numbers, some loss of precision occurs.

Forward motion is performed in steps of 100 mm. This means the robot cannot move an exact distance smaller than 100 mm. When approaching a waypoint, the maximum positional overshoot is therefore limited to approximately 50 mm (half a step). Similarly, rotational motion is limited to integer degrees, making it impossible to represent non-integer heading angles exactly.

Over longer paths, these small discretization effects may accumulate and cause deviation from the ideal continuous trajectory. To reduce this effect, the robot continuously tracks its position relative to the next node in the PRM path. Any small translational or angular overshoot is naturally corrected when aligning toward the following node.

Although this approach reduces mobility resolution, the resulting error remains bounded and acceptable for the scale of the simulated environment.

3.3 Path Planning

Robot path planning is divided into two phases: a *planning phase*, which analyses the map and constructs an obstacle-free network, and a *query phase*, which finds a path between a given start and goal using that network. Roadmap methods decouple these phases, so the roadmap is computed once and reused for any start and goal combination Corke, 2023.

The Probabilistic Roadmap (PRM) method builds the roadmap by randomly sampling free-space vertices and connecting them with obstacle-free straight-line edges, forming an embedded graph. With sufficiently many vertices, the graph spans the entire free space. The connectivity depends on the number of sampled points; too few vertices may result in disjoint components, preventing valid paths from being found.

Once the roadmap is constructed, the query phase finds a path by connecting the start position to the nearest roadmap vertex, traversing the graph via a minimum-distance route, and exiting at the vertex closest to the goal. The path is represented as an ordered list of via points:

$$\mathbf{p} = \{\mathbf{p}_0,\ \mathbf{p}_1,\ \ldots,\ \mathbf{p}_n\} \tag{8}$$

Key limitations of PRM include non-guaranteed connectivity, poor coverage of narrow passages, and non-smooth paths. Its primary advantage is computational efficiency: once planned, only the cheap query phase must be repeated when the start or goal changes, making PRM well suited to high-dimensional planning problems.

3.4 Localization and Mapping

Robot navigation requires two fundamental capabilities: knowing where the robot is (localization) and knowing what the environment looks like (mapping). These problems are deeply interdependent — a map is needed for accurate localization, while a reliable pose estimate is needed to build a map Corke, 2023.

Localization estimates the robot's pose $\mathbf{x}_v = (x_v,\, y_v,\, \theta_v)^T$ over time. *Dead reckoning* propagates the pose using odometry alone, but errors accumulate without bound. When a landmark map is available, sensor observations correct the estimate. The Extended Kalman Filter (EKF) linearizes the nonlinear motion and observation models via the Jacobian and is optimal for Gaussian noise. For non-Gaussian or multi-modal distributions, the Particle Filter (PF) maintains a set of weighted pose hypotheses at higher computational cost.

When the robot pose is assumed known, landmark positions can be estimated using an EKF. The state vector grows as new landmarks are observed:

$$\hat{\mathbf{x}} = (x_1,\, y_1,\, x_2,\, y_2,\, \cdots,\, x_M,\, y_M)^T \in \mathbb{R}^{2M\times 1} \tag{9}$$

Since landmarks are stationary, the prediction step is static ($\hat{\mathbf{x}}^+\langle k{+}1\rangle = \hat{\mathbf{x}}\langle k\rangle$). Observing one landmark provides no information about any other, so the covariance matrix remains block-diagonal.

When neither pose nor map is known, SLAM must be performed. EKF-SLAM augments the state vector to jointly estimate both:

$$\hat{\mathbf{x}} = (x_v,\, y_v,\, \theta_v,\, x_1,\, y_1,\, \cdots,\, x_M,\, y_M)^T \in \mathbb{R}^{(2M+3)\times 1} \tag{10}$$

The Kalman gain distributes innovation from each landmark observation across the full state vector, coupling pose and map estimates. However, the covariance matrix grows quadratically with the number of landmarks, making EKF-SLAM expensive for large environments. Rao-Blackwellized FastSLAM mitigates this by using a Particle Filter for pose estimation and an independent EKF per particle for mapping, reducing complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(M \log N)$. Graph-based SLAM represents the problem as a sparse pose graph and solves it as a least-squares optimization, and is the dominant paradigm in modern large-scale systems.

4 Results

5 Discussion

6 Conclusions

7 Bibliography

References

Li, H. (2017). Unicycle model of a wheeled mobile robot [Accessed: 2026-04-24]. https://www.researchgate.net/figure/Unicycle-model-of-a-wheeled-mobile-robot_fig54_319477982

Corke, P. (2023). Robotics, vision & control: Fundamental algorithms in python (3rd ed.). Springer Nature. https://doi.org/10.1007/978-3-031-06469-2